

مشروع جامعي - Compiler2

دليل مناقشة مرحلة Code Generation

Flask Extraction, Jinja Rendering, Watcher, Java HTTP
Server

مرجع عملي للأسئلة الشفهية، خريطة الملفات، وسيناريو العرض أمام اللجنة

إعداد من محتوى المشروع الحالي - نسخة RTL عربية

المحتويات

الجواب المختصر الذي تبدأ به أمام اللجنة

المخطط الكامل

خريطة الملفات التي يجب معرفتها

أين توجد الأشياء المهمة تحديداً؟

سيناريو المشروع الحالي خطوة بخطوة

1. ما الذي يصل إلى Code Generation؟

2. ماذا يفعل ContextExtractor أولاً؟

3. ما الفرق بين PyEval وExtractionCalls؟

4. ماذا يحدث مع Flask imports؟

5. ماذا يحدث مع app = Flask(__name__)؟

6. كيف تُستخرج store_name وproducts؟

7. ماذا يحدث مع product_count؟

8. كيف تُكتشف routes؟

9. ماذا تحتوي RouteInfo؟

10. ماذا يحدث مع app.run؟

11. كيف تُنفذ route؟

12. ماذا يحدث في index route؟

13. ماذا يحدث مع request.method؟

14. ماذا لو تعذر تنفيذ route كلها؟

15. ماذا تحتوي RenderJob؟

16. ماذا تحتوي ProjectContext؟

Jinja Code Generation باختصار دقيق

بنك الأسئلة والأجوبة

أسئلة المفهوم العام

أسئلة Context Extraction وPyEval

أسئلة Flask Routes وrender_template

أسئلة Jinja Rendering

أسئلة الخرج والتقارير

أسئلة Watcher

أسئلة Java Web Server وCRUD

أسئلة التصميم والدفاع عن المشروع

سيناريو العرض أمام اللجنة

خمس جمل إن ضاع منك التفصيل

قاعدة مهمة في الإجابة

هذا الدليل مخصص لشرح مرحلة Code Generation في مشروع Compiler2، وخصوصا جزء Flask، ثم صلته برندرة Jinja وJava HTTP Server والـ watcher.

الجواب المختصر الذي تبدأ به أمام اللجنة

مشروعنا لا يحول Python إلى machine code، بل يعمل كـ source-to-source compiler وstatic site generator. بعد نجاح Semantic Analysis، يأخذ `ContextExtractor` الـ Python AST ويستخدم `PyEval` لتنفيذ الجزء المدعوم من Python داخل Java. يستخرج `global variables` و `Flask routes`، ويحول كل `render_template` إلى `RenderJob` تحتوي اسم القالب والـ `context`. بعد ذلك يقرأ `JinjaRenderer` قالب Jinja، يحل inheritance و loops و conditions و expressions، ثم يكتب HTML النهائي داخل `output`. الـ `watcher` يعيد هذه العملية عند تعديل ملفات المصدر، بينما Java server يعرض الخرج ويدعم CRUD دائمة: يحدّث قائمة المنتجات داخل `app.py` ثم ينتظر الـ `watcher` ليعيد compilation كاملة من Lexer حتى HTML.

المخطط الكامل

```
app.py
-> FlaskLexer / FlaskParser
-> Python AST (ProgramNode)
-> Structural + Symbol + Scope + Semantic gates
-> ContextExtractor
    -> PyEval
    -> globals
    -> RouteInfo
    -> RenderJob
-> ProjectContext
-> JinjaRenderer
    -> HTML/Jinja Lexer + Parser
    -> HTML AST
    -> JinjaTemplateModel
    -> inheritance + expressions + loops + conditions
-> output/*.html
-> compiler_output/* reports
```

خريطة الملفات التي يجب معرفتها

الملف	وظيفته
<code>src/compilers/pipeline/GenerationPipeline.java</code>	المنسق الرئيسي لكل مراحل التوليد
<code>src/compilers/pipeline/GenerationResult.java</code>	النتيجة النهائية: exit code، الصفحات، diagnostics، ProjectContext والمسارات
<code>src/compilers/flask/generation/ContextExtractor.java</code>	استخراج globals و routes و RenderJobs من Python AST
<code>src/compilers/flask/generation/PyEval.java</code>	مفسر صغير للجزء المدعوم من Python AST داخل Java
<code>src/compilers/flask/generation/ProjectContext.java</code>	الحاوية النهائية لنتيجة استخراج Flask
<code>src/compilers/flask/generation/RouteInfo.java</code>	تمثيل route: endpoint، path، methods، function AST
<code>src/compilers/flask/generation/RenderJob.java</code>	مهمة رندرة: template، context، endpoint، output name
<code>src/compilers/html_css/render/JinjaRenderer.java</code>	تحويل template + context إلى HTML نصي
<code>src/compilers/html_css/render/JinjaTemplateModel.java</code>	تحويل HTML/Jinja AST إلى عقد سهلة للرندرة
<code>src/compilers/html_css/render/JinjaExprEvaluator.java</code>	تقييم <code>{{ expression }}</code> وشروط Jinja والفلاتر
<code>src/compilers/html_css/render/TemplateInheritance.java</code>	حل <code>extends</code> واختيار blocks التي تعمل override
<code>src/compilers/report/GenerationLog.java</code>	بناء generation log بترتيب المراحل
<code>src/compilers/report/AstJsonWriter.java</code>	كتابة Python AST و Jinja AST كـ JSON
<code>src/compilers/report/SemanticReportWriter.java</code>	كتابة semantic report
<code>src/compilers/report/GenerationReportWriter.java</code>	كتابة dashboard باسم <code>report.html</code>
<code>src/compilers/pipeline/ProjectWatcher.java</code>	مراقبة ملفات المصدر وتشغيل compilation كاملة عند الحفظ
<code>src/compilers/server/CompilerWebServer.java</code>	عرض ملفات output واستقبال CRUD وانتظار نتيجة watcher

الملف	وظيفته
<code>src/compilers/server/PythonSourceCollectionWriter.java</code>	إيجاد assignment القائمة عبر Parser وتحديثها داخل app.py كتابة ذرية
<code>src/Main/LiveProjectMain.java</code>	تشغيل server + watcher + compiler معا
<code>src/Main/WatchProjectMain.java</code>	تشغيل compiler + watcher دون web server
<code>src/Main/UnifiedMain.java</code>	CLI وتشغيل GenerationPipeline مرة واحدة

أين توجد الأشياء المهمة تحديداً؟

السؤال	المكان في الكود
أين تبدأ Code Generation؟	<code>GenerationPipeline.run()</code>
أين يبدأ Flask extraction؟	<code>GenerationPipeline</code> عند إنشاء <code>ContextExtractor</code> ، قرب السطر 180
أين تُخزن المتغيرات العامة؟	<code>ContextExtractor.moduleEnv</code> ، قرب السطر 74
أين تُنفذ assignments؟	<code>PyEval.execAssignment()</code> ، قرب السطر 247
أين تُخزن القيمة باسم المتغير؟	<code>PyEval.store()</code> ، قرب السطر 335
أين تُقرأ قيمة identifier؟	<code>PyEval.lookup()</code> ، قرب السطر 509
أين تتحول Python list/dict إلى Java؟	<code>PyEval.eval()</code> ، قرب السطر 391
أين تُسجل functions؟	<code>ContextExtractor.registerFunction()</code> ثم <code>PyEval.registerFunction()</code>
أين تُكتشف decorators وال routes؟	<code>ContextExtractor.tryRoute()</code> ، قرب السطر 197
أين تُنفذ route functions؟	<code>ContextExtractor.evaluateRoute()</code> ، قرب السطر 250
أين يُعالج <code>request.method</code> ؟	<code>PyEval.condition()</code> في lenient route mode ، قرب السطر 277
أين يتم fallback إذا تعذر تنفيذ route؟	<code>ContextExtractor.staticFallback()</code> ، قرب السطر 314
أين تُعرض <code>render_template</code> ؟	<code>ContextExtractor.ExtractionCalls.intercept()</code> ، قرب السطر 381
أين تُعرض <code>Flask()</code> و <code>url_for()</code> ؟	نفس <code>ExtractionCalls.intercept()</code>
أين يُتجاهل <code>app.run()</code> ؟	<code>ExtractionCalls.interceptMethod()</code> ، قرب السطر 423
أين تتكون ProjectContext؟	نهاية <code>ContextExtractor.extract()</code> ، قرب السطر 112
أين يتحول اسم template إلى output name؟	<code>RenderJob.getOutputFileName()</code>
أين تبدأ Jinja rendering؟	<code>JinjaRenderer.render()</code> ، قرب السطر 77
أين يُقرأ ويُحلل ملف template؟	<code>JinjaRenderer.load()</code> ، قرب السطر 294
أين تنفذ <code>{{ value }}</code> ؟	<code>JinjaExprEvaluator + JinjaRenderer.renderNode()</code>

السؤال	المكان في الكود
أين تنفذ <code>{% if %}</code> ؟	<code>JinjaRenderer.renderIf()</code> ، قرب السطر 169
أين تنفذ <code>{% for %}</code> ؟	<code>JinjaRenderer.renderFor()</code> ، قرب السطر 192
أين تُحل <code>template inheritance</code> ؟	<code>TemplateInheritance.resolve()</code>
أين يُحل <code>url_for</code> ؟	<code>JinjaRenderer.resolveUrl()</code> و <code>GenerationPipeline.routeHrefs()</code>
أين يُكتب HTML ؟	<code>GenerationPipeline</code> بعد <code>renderer.render()</code> ، قرب السطر 204
أين تُنسخ CSS/JS/assets ؟	<code>GenerationPipeline.copySupportFiles()</code> ، قرب السطر 354
أين تُكتب التقارير ؟	<code>GenerationPipeline.finish()</code> ، قرب السطر 454
أين يعمل <code>watcher</code> ؟	<code>ProjectWatcher.watch()</code>
أين يبدأ السيرفر ؟	<code>CompilerWebServer.start()</code>
أين توجد CRUD ؟	<code>CompilerWebServer.handleCollectionRequest()</code>
أين يُعدّل <code>app.py</code> ؟	<code>PythonSourceCollectionWriter.write()</code>
أين ينتظر السيرفر إعادة التوليد ؟	<code>CompilerWebServer.awaitRebuild()</code>

سيناريو المشروع الحالي خطوة بخطوة

1. ما الذي يصل إلى Code Generation؟

يصل `ProgramNode` صالح بعد Semantic Analysis. إذا وجد `DiagnosticReporter` أي Error، تتوقف العملية قبل Context Extraction. ال warning وحده لا يمنع التوليد.

2. ماذا يفعل ContextExtractor أولاً؟

ينشئ `moduleEnv` ويضع:

```
__name__ -> "__main__"
__package__ -> ""
```

ثم ينشئ `PyEval(moduleEnv, new ExtractionCalls())`.

3. ما الفرق بين PyEval وExtractionCalls؟

`PyEval` ينفذ Python العام مثل assignment و if و for والدوال والعمليات. `ExtractionCalls` هي interceptor للأشياء الخاصة بـ Flask مثل `Flask()` و `render_template()` و `url_for()` و `app.run()`.

4. ماذا يحدث مع Flask imports؟

لا يتم استيراد Flask الحقيقي. تُسجل الأسماء كـ `UnsupportedRef`، وعندما يتم استدعاؤها يرسلها `PyEval` إلى `ExtractionCalls`.

5. ماذا يحدث مع app = Flask(__name__)؟

يعترض `ExtractionCalls` استدعاء `Flask` ويرجع `FlaskApp` marker. تُخزن:

```
app -> FlaskApp marker
```

لا يتم تشغيل Flask الحقيقي.

6. كيف تُستخرج store_name وproducts؟

ينفذ `PyEval` assignments. يتحول Python string إلى Java `String`، وتتحول Python list إلى `ArrayList`، وكل dict إلى `LinkedHashMap`.

```
store_name -> "Products Store"
products   -> List<Map>[3]
```

7. ماذا يحدث مع product_count؟

يسجل `PyEval` الدالة كـ `UserFunction` ولا ينفذها عند تعريفها. عند الوصول لاحقاً إلى `product_count(products)` ينشئ `local frame`، ينفذ `loop` و `return`، فتكون النتيجة 3.

8. كيف تُكتشف routes؟

يفحص `ContextExtractor.tryRoute()` decorators من أشكال:

```
@app.route("/", methods=["GET"])
@app.get("/path")
@app.post("/path")
```

وينشئ `RouteInfo` لكل `route`.

9. ماذا تحتوي RouteInfo؟

```
endpoint -> اسم view function
path      -> مسار HTTP
methods   -> GET/POST
function  -> FunctionDefNode الخاص بال view
```

10. ماذا يحدث مع app.run؟

يتم تقييم شرط `__name__ == "__main__"`، لكن `app.run()` يُعرض ويصبح `no-op`. هدف `compilation` استخراج الصفحات، لا تشغيل `Flask server`.

11. كيف تُنفذ route؟

في المرور الثاني، يستدعي `ContextExtractor.evaluateRoute()` كل view كدالة بدون arguments داخل local frame مستقل، ثم يراقب أي `render_template` تحدث.

12. ماذا يحدث في index route؟

يقيّم:

```
render_template(
    "index.jinja",
    title=store_name,
    products=products,
    count=product_count(products)
)
```

فتصبح القيم:

```
template = index.jinja
title     = Products Store
products  = List[3]
count     = 3
```

ثم ينشئ `RenderJob`.

13. ماذا يحدث مع request.method؟

لا يوجد HTTP request أثناء static compilation. إذا لم يستطع `PyEval` تقييم شرط route مثل `request.method == "POST"`، يسجل warning ويعامل الشرط كـ `False`، وبذلك يولد مسار GET الافتراضي.

14. ماذا لو تعذر تنفيذ route كلها؟

يستخدم `staticFallback()`، يبحث في AST عن آخر `render_template`، ويقيّم ما يمكن تقييمه. أي context يعتمد على request يُحذف مع warning. إذا لم يكن اسم القالب قابلاً للتحديد، يصدر Code Generation Error.

15. ماذا تحتوي RenderJob؟

```
templateName
context Map<String,Object>
endpoint
line / column
outputFileName
```

`index.jinja` تتحول إلى `index.html` بواسطة `getOutputFileName()`.

16. ماذا تحتوي ProjectContext؟

```
globals
routes
renderJobs
extraction log lines
```

هذه هي نقطة التسليم بين Flask Code Generation و Jinja Code Generation.

Jinja Code Generation باختصار دقيق

1. يبنّي `GenerationPipeline` خريطة endpoint إلى href.
2. ينشئ `JinjaRenderer(reporter, templateRoot, routeHrefs)`.
3. يمر على كل `RenderJob`.
4. يقرأ القالب من `templates/`.
5. يشغل HTML/Jinja lexer وparser ويبني HTML AST.
6. يحول AST إلى `JinjaTemplateModel`.
7. يحل سلسلة `{% extends %}`.
8. يضع context في frame داخل `JinjaExprEvaluator`.
9. يرندر Text وOutput وIf وFor وBlock nodes.
10. يعيد HTML ك String.
11. يكتب `output/<page>.html` بترميز UTF-8.

بنك الأسئلة والأجوبة

أسئلة المفهوم العام

س1: ما المقصود بـ Code Generation في مشروعكم؟

ج: تحويل Flask/Python AST مع Jinja templates إلى صفحات HTML ثابتة وتقارير، وليس توليد machine code.

س2: ما مدخل ومخرج قسم Flask؟

ج: المدخل `ProgramNode` صالح دلاليًا، والمخرج `ProjectContext` تحتوي `globals` و `routes` و `RenderJobs`.

س3: هل تشغلون Python أو Flask الحقيقي؟

ج: لا. `PyEval` يفسر subset من Python AST داخل Java، و `ExtractionCalls` تعترض عمليات Flask المهمة.

س4: لماذا لم تشغلوا Flask الحقيقي؟

ج: لأن المطلوب compiler deterministic يستخرج صفحات ثابتة. تشغيل Flask الحقيقي يحتاج Python runtime و request lifecycle وسيحول المرحلة إلى تشغيل تطبيق، لا Code Generation.

س5: من هو المنسق الرئيسي؟

ج: `GenerationPipeline`. يرتب Jinja rendering، context extraction، semantic gates، parse، نسخ الدعم وكتابة التقارير.

س6: متى تتوقف Code Generation؟

ج: تتوقف قبل extraction عند semantic errors، وتتوقف برمز 2 عند extraction أو template structural errors. Warnings لا تمنع الخرج.

أسئلة Context Extraction و PyEval

س7: أين استخراج المتغيرات العامة؟

ج: في `ContextExtractor.executeTopLevel()` باستخدام `PyEval.exec()`، وتُحفظ القيم في `ContextExtractor.moduleEnv`.

س8: أين ينفذ assignment؟

ج: `PyEval.execAssignment()` يحسب RHS بواسطة `eval()` ثم يخزنها بواسطة `store()`.

س9: كيف تتحول قيم Python إلى Java؟

ج: `int` إلى `BigInteger` ، `float` إلى `Double` ، `string` إلى `String` ، `None` إلى `null` ، `list/tuple` إلى `ArrayList` ، `dict` إلى `LinkedHashMap` ، `set` إلى `LinkedHashSet`.

س10: لماذا تستخدمون LinkedHashMap؟

ج: للحفاظ على ترتيب الإدخال، وهذا يجعل context والخرج والتقارير deterministic.

س11: كيف تعمل scopes أثناء التقييم؟

ج: `PyEval` يملك stack من Frames. كل function call يدفع local frame، والبحث يبدأ محلياً ثم يصل إلى module environment، وبعد return يُزال frame.

س12: هل يستخدم PyEval الـ SymbolTable؟

ج: لا. `SymbolTable` استُخدمت في Semantic Analysis. التقييم يستخدم frame chain وقيم Java فعلية، حتى تبقى حدود التوليد بسيطة.

س13: ما أوامر Python المدعومة؟

ج: assignments، if/for/while، return/break/continue، functions، arithmetic، comparisons، lists/dicts/subscripts وعدد من built-ins و methods.

س14: كيف تمنعون loop لا نهائية أثناء التوليد؟

ج: `PyEval` يحدد 200000 evaluation steps، و 64 call depth، و 100000 عنصر كحد أقصى لـ `range`.

س15: ماذا يحدث لشيء Python غير مدعوم؟

ج: يرمي `PyEval.EvalError`. يحوله `ContextExtractor` إلى diagnostic أو يستخدم static fallback إذا كانت المشكلة داخل route.

س16: أين تُسجل functions؟

ج: `ContextExtractor.registerFunction()` يستدعي `PyEval.registerFunction()`، فتُحفظ `FunctionDefNode` وال default values.

س17: متى تُحسب default arguments؟

ج: وقت تسجيل function، مثل سلوك Python في حساب defaults وقت التعريف.

أسئلة Flask Routes وrender_template

س18: كيف تعرفون أن function هي route؟

ج: `tryRoute()` يفحص decorator ويبحث عن method باسم `route` أو `get` أو `post`، ثم يقيّم path وmethods.

س19: ما هو endpoint؟

ج: اسم view function افتراضياً، مثل `index` أو `add_product`، ويستخدم في `url_for`.

س20: أين تحفظ routes؟

ج: في `ContextExtractor.routes` كـ `Map<String, RouteInfo>` مرتبة حسب التعريف.

س21: كيف تتحول render_template إلى RenderJob؟

ج: يعترضها `ExtractionCalls.intercept()`. يأخذ أول positional argument كاسم template، وال keyword arguments كـ context، ويربط المهمة بـ `currentEndpoint`.

س22: لماذا اسم template يجب أن يكون قابلاً للتقييم؟

ج: لأن المولد يجب أن يعرف أي ملف يقرأ أثناء compilation. اسم يعتمد كلياً على request لا يمكن تحويله بأمان إلى صفحة ثابتة.

س23: ماذا لو route لا ترندر template؟

ج: تُسجل أنها skipped ولا تُنشأ لها صفحة.

س24: ماذا لو اكتشف أكثر من render_template في route؟

ج: يسجل note ويحتفظ بآخر RenderJob تم التقاطها لمسار GET الذي تم تقييمه.

س25: لماذا request.method يصبح False؟

ج: لا يوجد request أثناء static generation. في lenient route mode، الشرط غير القابل للتقييم يسجل warning ويعامل كـ False لاختيار view GET.

س26: أين تُحذف markers من globals؟

ج: `ContextExtractor.publicGlobals()` يستبعد `ModuleRef` و `FlaskApp` و `UnsupportedRef` و `__name__` و `__package__`.

س27: ما الفرق بين `RenderJob` و `RouteInfo`؟

ج: `RouteInfo` تصف HTTP route ودالتها. `RenderJob` تصف صفحة يجب رندرتها مع `template` و `context`.

س28: ما فائدة `ProjectContext`؟

ج: عقد واضح بين نصف Flask ونصف Jinja، ويستخدمه أيضا Java server كـ `successful compilation snapshot`.

أسئلة Jinja Rendering**س29: أين تبدأ الرندرة الفعلية؟**

ج: `GenerationPipeline` يستدعي `JinjaRenderer.render(templateName, context)` لكل `RenderJob`.

س30: هل تتعاملون مع Jinja كنص فقط؟

ج: لا. يتم `lex/parse` لل قالب وبناء `HTML/Jinja AST`، ثم `JinjaTemplateModel`، وبعدها `tree rendering`.

س31: كيف تعمل `template inheritance`؟

ج: `TemplateInheritance.resolve()` يبني `chain` من `child` إلى `base`. يبدأ الرسم من `base tree`، وعند `block` يبحث `lookupBlock()` عن أقرب `override` من `child`.

س32: كيف تعمل `{{ title }}`؟

ج: `renderNode()` يرسل `expression` إلى `JinjaExprEvaluator`، يحول القيمة إلى نص، يطبق `HTML` `escaping`، ثم يضيفها إلى `StringBuilder`.

س33: كيف تعمل `{% for product in products %}`؟

ج: `renderFor()` يقيّم `iterable`، ثم ينشئ `frame` لكل عنصر يحتوي `product` و `loop.index/index0/first/last/length` ويرندر `body`.

س34: كيف تعمل `{% if %}` ؟

ج: `renderIf()` يقيّم الفروع بالترتيب مستخدماً Python/Jinja truthiness ويرندر أول branch نتيجه True، وإلا يرندر else.

س35: كيف يصل `product.name` إلى dict ؟

ج: `JinjaExprEvaluator` يتعامل مع attribute access على Map كمفتاح باسم `name`.

س36: ما الفلاتر المدعومة؟

ج: `default` و `upper` و `lower` و `title` و `length`.

س37: هل تطبقون HTML escaping ؟

ج: نعم افتراضياً في `JinjaExprEvaluator.escapeHtml()` لحماية `' < > "`.

س38: كيف يعمل `url_for` ؟

ج: بيني Pipeline خريطة endpoint إلى generated filename. `url_for('index')` يصبح `index.html` ، و `url_for('static', filename='style.css')` يصبح اسم الملف.

س39: ماذا يحدث إذا كان endpoint مجهولاً؟

ج: يصدر Jinja value warning، ويستمر توليد الصفحة بقيمة فارغة لذلك الموضع.

س40: ما الفرق بين Jinja value error و structural error ؟

ج: value error مثل variable غير معرّف هو warning ويُترك output فارغاً. structural error مثل blocks غير متوازنة أو template مفقود يمنع كتابة الصفحة.

أسئلة الخرج والتقارير

س41: أين تُكتب الصفحة؟

ج: في `GenerationPipeline` بعد عودة HTML String، باستخدام `outputDir.resolve(job.getOutputFileName())` و UTF-8.

س42: كيف تمنعون templateين من الكتابة فوق نفس الملف؟

ج: `validateOutputNames()` يفحص كل RenderJobs قبل الرندرة ويصدر error عند collision.

س43: لماذا تحذف الصفحات القديمة أولاً؟

ج: `cleanGeneratedPages()` يزيل HTML المولّد القديم حتى لا تبقى صفحة stale بعد تغيير routes أو فشل `.render`.

س44: ماذا يُنسخ إلى output؟

ج: `app.py` و `style.css` و `script.js`، مع mirror recursive لمجلدات `templates` و `static` و `assets`.

س45: ما التقارير الناتجة؟

ج: `ast_python.json` و `ast_jinja.json` و `semantic_report.txt` و `generation_log.txt` و `report.html`.

س46: ما exit codes؟

ج: 0 نجاح، 1 فشل 2، frontend/semantic، فشل 3، extraction/rendering، خطأ CLI أو تشغيل داخلي عند نقطة الدخول.

أسئلة Watcher**س47: ما وظيفة watcher؟**

ج: مراقبة ملفات المصدر وتشغيل `new GenerationPipeline().run(paths)` عند أي تغيير مدعوم.

س48: ماذا يراقب؟

ج: `py/.html/.jinja/.j2/.css/.js` والصور والخطوط، ومجلدات `templates/static/assets` بشكل recursive.

س49: لماذا يتجاهل output و compiler_output؟

ج: لأن compiler يكتب فيهما. مراقبتهم تسبب feedback loop وإعادة توليد لا نهائية.

س50: لماذا debounce لمدة 300ms؟

ج: IntelliJ قد يولد عدة file events لعملية Save واحدة؛ debounce يجمعها في rebuild واحدة.

س51: من أين يعيد watcher المراحل؟

ج: يعيد pipeline كاملة من Python Lexer حتى HTML، لأن أي source change قد يؤثر في AST أو semantics أو context أو القالب.

س52: هل watcher يعمل browser refresh؟

ج: لا. يعيد الملفات فقط. المستخدم يعمل refresh، ولا يوجد SSE أو WebSocket.

أسئلة Java Web Server وCRUD**س53: هل Java server هو Flask server؟**

ج: لا. هو `com.sun.net.httpserver.HttpServer` يعرض الملفات المولدة ويقدم API صغيرة. لا يشغل `app.py` ولا ينفذ `Flask request lifecycle`.

س54: ماذا يفعل GET للصفحات؟

ج: `handleStaticFile()` يقرأ الملف من `output`، يتحقق من `path traversal`، يحدد `Content-Type`، ويرسله مع `Cache-Control: no-store`.

س55: ما API المتاحة؟

ج:

```
GET    /api/products
GET    /api/products/{id}
POST   /api/products
PUT    /api/products/{id}
DELETE /api/products/{id}
```

س56: أين تُحفظ تعديلات الواجهة؟

ج: تُكتب فعلياً داخل `assignment` القائمة في `app.py`. أما `liveGlobals` فهي نسخة قراءة من آخر `compilation` ناجحة يستخدمها `GET` والتحقق من النتيجة.

س57: هل PUT/DELETE يعدلان app.py؟

ج: نعم. يبني السيرفر القائمة الجديدة ثم يستدعي `PythonSourceCollectionWriter.write()` لاستبدال قيمة القائمة فقط داخل الملف.

س58: هل POST/PUT/DELETE تعيد Lexer وParser؟

ج: نعم. لأنها أصبحت تعدل `source` الحقيقي، يلتقط `watcher` تغيير `app.py` ويعيد `Lexer` ثم `Parser` و `AST` و `Semantic` و `Jinja Context` حتى `HTML`.

س59: كيف تختارون المنتج؟

ج: كل product يملك `id`. المسار `/api/products/{id}` يبحث عن Map التي يساوي حقل `id` فيها `id` المطلوب.

س60: كيف تمنعون حالة بيانات فاسدة إذا فشلت الرندرة؟

ج: لا يغير السيرفر live snapshot مباشرة. الكاتب يبني الملف كاملاً ويفحصه بال Parser قبل `atomic replace`. وبعد الحفظ ينتظر السيرفر compilation ناجحة؛ وعند الفشل يحتفظ `acceptCompilation()` بآخر snapshot ناجحة ويعيد HTTP error.

س61: ماذا يفعل PythonSourceCollectionWriter؟

ج: يحلل `app.py` بغرامر المشروع، يجد top-level assignment الذي يطابق اسم القائمة، يولد Python literal للقيم الجديدة، يستبدل RHS فقط، يعيد فحص الملف، ثم يكتبه UTF-8 بملف مؤقت و `atomic move`.

س62: لماذا stateLock و mutationLock؟

ج: `stateLock` يحمي compilation snapshot و liveGlobals والتواصل مع callback ال watcher. و `mutationLock` يمنع طلبي CRUD من الكتابة على `app.py` معاً.

س63: كيف يعرف طلب CRUD أن إعادة التوليد انتهت؟

ج: يحفظ رقم آخر compilation قبل الكتابة ثم ينتظر `acceptCompilation(result)`. عندما تصل نتيجة ناجحة وتساوي القائمة المستخرجة القيم المطلوبة، يكمل الطلب ويرسل redirect أو 204. توجد مهلة زمنية ورسالة خطأ واضحة إذا لم يؤكد watcher.

س64: كيف تعمل صفحة Edit؟

ج: رابط Edit يحمل id في data attribute. `script.js` يفتح `edit_product.html?id=...`، ثم GET يجلب المنتج، و submit يرسل PUT، وبعد النجاح تعود الصفحة إلى index.

س65: كيف يعمل Delete؟

ج: `script.js` يطلب تأكيداً ويرسل DELETE بال id. السيرفر يحذف العنصر من نسخة القائمة، يكتبها في `app.py`، ينتظر full rebuild، ثم يعود المتصفح إلى index.

أسئلة التصميم والدفاع عن المشروع

س66: لماذا ProjectContext بيانات Java عادية؟

ج: لفصل Flask extraction عن Jinja renderer. الحد بينهما هو String/Number/ Boolean/null/List/Map دون ربط renderer بعقد Python AST.

س67: لماذا تحتفظون بآخر successful snapshot؟

ج: حتى لا يستبدل compilation فاشل حالة السيرفر ببيانات ناقصة. `acceptCompilation` يقبل النتيجة فقط إذا كانت ناجحة ولها ProjectContext.

س68: هل المشروع يدعم Flask كله؟

ج: لا، يدعم subset تعليمي واضح. أي شيء خارج subset ينتج diagnostic أو warning بدلا من قيمة صامتة خاطئة.

س69: ما أهم حدود المشروع؟

ج: لا يوجد Flask runtime حقيقي، ولا database persistence، ولا route parameters متقدمة، ولا auto browser reload. الاستمرارية الحالية file-based داخل assignment قائمة واضحة في `app.py` وليست قاعدة بيانات عامة.

س70: ما الاختبارات التي تثبت Code Generation؟

ج: `ContextExtractionHarness` لاستخراج Flask، `JinjaRenderHarness` للرنديرة، `GenerationEndToEndHarness` للخروج والتقارير، `ProjectWatcherHarness` للمراقبة، و `CompilerWebServerHarness` لطلبات HTTP وCRUD. المجموع الحالي 112 اختبارا.

سيناريو العرض أمام اللجنة

1. شغل `Main.LiveProjectMain` من IntelliJ.
2. انتظر `Run #1 - INITIAL BUILD` و `RESULT: SUCCESS`.
3. افتح `http://localhost:8080/index.html` وشاهد 3 products.
4. عدل سعر منتج في `Tests/generation/sample_project/app.py` واعمل Save.
5. اشرح أن watcher اكتشف الملف وأعاد HTML -> Lexer.
6. اعمل Refresh وشاهد السعر الجديد.
7. أضف منتجا من الواجهة وافتح `app.py` لتري أن القائمة تغيرت فعليا.
8. اشرح رسائل watcher: اكتشاف Python ثم HTML -> Lexer ثم نجاح الطلب.
9. عدل المنتج واحذفه و اشرح PUT/DELETE بنفس المسار.
10. أضف منتجا أخيرا، أوقف السيرفر وأعد تشغيله لتثبت أن التعديل بقي في `app.py`.

خمس جمل إن ضاع منك التفصيل

1. `GenerationPipeline` هو المنسق الرئيسي.
2. `ContextExtractor + PyEval` يحولان Python AST إلى `ProjectContext`.
3. كل `render_template` تصبح `RenderJob` فيها `template` و `context`.
4. `JinjaRenderer` يحول `RenderJob` إلى HTML ويكتبها Pipeline داخل `output`.
5. watcher يعيد compilation عند `source save`، والسيرفر يعرض الخرج ويعمل CRUD دائمة بكتابة القائمة في `app.py` ثم انتظار `full rebuild`.

قاعدة مهمة في الإجابة

إذا سألك عن أي وظيفة، ابدأ بهذا النمط:

المسؤول هو `<ClassName>` داخل `<path>`. الدالة الأساسية هي `<method>`. تستلم `<input>`، تنفذ `<behavior>`، وتنتج `<output>`.

مثال:

المسؤول عن استخراج المتغيرات هو `ContextExtractor` داخل `src/compilers/flask/generation/ContextExtractor.java`. يمر على `top-level statements` بواسطة `executeTopLevel()`، ويستخدم `PyEval` لتنفيذ `assignments`، ثم يحفظ القيم في `moduleEnv` ويضع القيم العامة النهائية داخل `ProjectContext`.